

Lab1: DataLab 实验报告

姓名：袁锐 学号：25301020123

环境与测试结果

实验在 WSL Ubuntu 24.04 中完成，编译后运行规则检查和 btest。19 个函数均通过规则检查，btest 各项 Errors 为 0，最终结果为 110/110。

```
make -B all
python3 check_ops.py bits.c
./btest
```

规则检查：

```
+ python3 check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 48/48 operators
mul5Sat: 25/30 operators
classifyAdd3: 15/52 operators
floatScaleThreeHalves: 43/60 operators
floatRoundEven: 27/65 operators
float_i2f: 33/40 operators
bitCount: 29/40 operators
bitReverse: 28/34 operators
All 19 functions passed operator checks.
```

check_ops 运行结果

正确性测试：

```
+ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

btest 运行结果

各题实现

P1 signMask

```
int signMask(void) {
    return 1 << 31;
}
```

将 1 左移 31 位即可得到只有最高位为 1 的掩码 0x80000000。

P2 bitXor

```
int bitXor(int x, int y) {
    return ~(~(x & ~y) & ~(~x & y));
}
```

分别取出 x 为 1、y 为 0，以及 x 为 0、y 为 1 的位置；这两组互斥位合起来就是异或。题目只允许 ~ 和 &，所以用德摩根律替代按位或。

P3 negativePart

```
int negativePart(int x){
    int sign = x >> 31;
    return ((x ^ sign) + 1) & sign;
}
```

x 的算术右移结果是符号掩码：非负数为 0，负数为全 1。与 x 异或后加 1，负数得到相反数；最后与符号掩码相与，非负分支清零。

P4 copyByteWithin

```
int copyByteWithin(int x, int src, int dst) {
    int srcShift = src << 3;
    int dstShift = dst << 3;
    int byte = (x >> srcShift) & 0xff;
    int mask = 0xff << dstShift;
    return (x & ~mask) | (byte << dstShift);
}
```

字节编号左移 3 位得到位偏移。先右移并用 0xff

取出源字节，再清空目标字节位置后写入；其他字节保持原样，源和目标相同时结果也不变。

P5 logicalShift

```
int logicalShift(int x, int n) {
    int mask = ~(((1 << 31) >> n) << 1);
    return (x >> n) & mask;
}
```

先算术右移，再构造只保留低 32-n 位的掩码以清除符号扩展。n 为 0 时掩码全 1，n 为 31 时只留下移到最低位的原符号位。

P6 swapNibblePairs

```
int swapNibblePairs(int x) {
    int mask = 0x0f | (0x0f << 8);
    mask = mask | (mask << 16);
    return ((x & mask) << 4) | ((x >> 4) & mask);
}
```

重复构造 0x0f 掩码，分别取出每个字节的低半字节和高半字节，再向相反方向移动 4 位并合并。

P7 secondLowestZeroBit

```
int secondLowestZeroBit(int x) {
    int zeros = ~x;
    int first = zeros & (~zeros + 1);
    int rest = zeros ^ first;
    return rest & (~rest + 1);
}
```

先对 x 取反，让原来的 0 位变成 1。v & (~v + 1) 可隔离最低位的 1：第一次找到最低 0 位，清掉它后再做一次，就得到第二低的 0 位；若不足两个 0 位，结果为 0。

P8 oddParity

```
int oddParity(int x) {
    x = x ^ (x >> 16);
    x = x ^ (x >> 8);
    x = x ^ (x >> 4);
    x = x ^ (x >> 2);
    x = x ^ (x >> 1);
    return !(x & 1);
}
```

依次异或相距 16、8、4、2、1 位的比特，把 32 位的奇偶信息折叠到最低位。最低位为 0 表示 1 的个数为偶数，故用逻辑非得到题目要求的返回值。

P9 rotateRightBits

```
int rotateRightBits(int x, int n) {
    int shift = n & 31;
    int left = (32 + (~shift + 1)) & 31;
    int mask = ~(((1 << 31) >> shift) << 1);
    return ((x >> shift) & mask) | (x << left);
}
```

$n \& 31$ 将位移量化为 0–31。右移部分用掩码清除算术右移补入的符号位，左移部分把移出的位绕回高端； n 是 32 的倍数时返回原值。

P10 roundEvenPow2

```
int roundEvenPow2(int x, int n) {
    int mask = (1 << n) + ~0;
    int remainder = x & mask;
    int half = 1 << (n + ~0);
    int greater = ((half + (~remainder + 1)) >> 31) & 1;
    int equal = !(remainder ^ half);
    int increment = greater | (equal & ((x >> n) & 1));
    return (x + (increment << n)) & ~mask;
}
```

低 n 位给出余数，余数与 $2^{(n-1)}$

比较后决定向下或向上舍入。正好在中点时检查当前商的最低位：商为奇数才进位，这样结果商为偶数。

P11 midpointTowardFirst

```
int midpointTowardFirst(int x, int y) {
    int common = x & y;
    int different = x ^ y;
    int average = common + (different >> 1);
    int xSign = x >> 31;
    int ySign = y >> 31;
    int difference = x + (~y + 1);
    int sameSign = ~(xSign ^ ySign);
    int greater = (((~xSign & ySign) |
        (sameSign & !(difference >> 31))) & 1);
    return average + ((different & 1) & greater);
}
```

用 $(x \& y) + ((x \wedge y) \gg 1)$ 计算向下取整的中点，避免直接相加溢出。两数奇偶不同且 x 大于 y 时加 1，选择更接近第一个参数的整数；同号时用差值判断大小，异号时由符号位判断。

P12 isBetweenEitherOrder

```
int isBetweenEitherOrder(int x, int a, int b) {
    int d1 = a + (~x + 1);
    int s1 = a ^ x;
    int leAx = (((s1 & a) | (~s1 & d1)) >> 31) | !d1;
    int d2 = x + (~b + 1);
    int s2 = x ^ b;
    int leXb = (((s2 & x) | (~s2 & d2)) >> 31) | !d2;
    int d3 = b + (~x + 1);
    int s3 = b ^ x;
    int leBx = (((s3 & b) | (~s3 & d3)) >> 31) | !d3;
    int d4 = x + (~a + 1);
    int s4 = x ^ a;
    int leXa = (((s4 & x) | (~s4 & d4)) >> 31) | !d4;
    return ((leAx & leXb) | (leBx & leXa)) & 1;
}
```

分别判断 $a \leq x$ 、 $x \leq b$ 、 $b \leq x$ 和 $x \leq a$ ，再组合成两种端点顺序的闭区间判断。比较时先看符号是否相异，只有同号时才依赖差值符号，避免异号相减溢出；相等端点也算在区间内。

P13 mul5Sat

```
int mul5Sat(int x) {
    int four = x << 2;
    int product = four + x;
    int recovered = four >> 2;
    int overflowFour = ((!(recovered ^ x)) << 31) >> 31;
    int overflowAdd = ((four ^ product) & ~(four ^ x)) >> 31;
    int overflow = overflowFour | overflowAdd;
    int sign = x >> 31;
    int signBit = 1 << 31;
    int maxValue = ~signBit;
    int saturated = (sign & signBit) | (~sign & maxValue);
    return (overflow & saturated) | (~overflow & product);
}
```

将 $5x$ 拆成 $x \ll 2$ 再加 x ，分别检查左移和加法溢出。发生溢出时根据 x 的符号选 `INT_MIN` 或 `INT_MAX`，否则返回乘积。

P14 classifyAdd3

```
int classifyAdd3(int x, int y, int z) {
    int mask = (1 << 30) + ~0;
    int lowX = x & mask;
    int lowY = y & mask;
    int lowZ = z & mask;
    int highX = x >> 30;
    int highY = y >> 30;
    int highZ = z >> 30;
    int lowXY = lowX + lowY;
    int carryXY = lowXY >> 30;
    lowXY = lowXY & mask;
    int lowXYZ = lowXY + lowZ;
    int carryXYZ = lowXYZ >> 30;
    int carry = carryXY + carryXYZ;
    int high = highX + highY + highZ + carry;
    int positiveOverflow = (! (high >> 31)) & (!(high >> 1));
    int negativeOverflow = ((high + 2) >> 31) & 1;
    return positiveOverflow | (~negativeOverflow + 1);
}
```

把每个输入拆成低 30 位和高段，先累加低段并记录进位，再把进位加到高段。高段大于 1 表示正溢出，小于 -2 表示负溢出，其余结果都能用 32 位有符号数表示。

P15 floatScaleThreeHalves

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exponent = (uf >> 23) & 0xffu;
    unsigned fraction = uf & 0x7fffffu;
    unsigned product;
    unsigned rounded;
    unsigned shift;
    unsigned remainder;
    unsigned half;
    unsigned mask;

    if (exponent == 0xffu)
        return uf;

    if (exponent == 0) {
        product = fraction * 3u;
        rounded = product >> 1;
        remainder = product & 1u;
        if (remainder && (rounded & 1u))
            rounded++;
        if (rounded >= 0x8000000u)
            return sign | 0x00800000u | (rounded - 0x8000000u);
        return sign | rounded;
    }

    product = (0x8000000u | fraction) * 3u;
    shift = (product & 0x02000000u) ? 2u : 1u;
    exponent += shift - 1u;
    rounded = product >> shift;
    mask = (1u << shift) - 1u;
    remainder = product & mask;
    half = 1u << (shift - 1u);
    if (remainder > half || (remainder == half && (rounded & 1u)))
        rounded++;
    if (rounded == 0x10000000u) {
        rounded >>= 1;
        exponent++;
    }
    if (exponent >= 0xffu)
        return sign | 0x7f800000u;
    return sign | (exponent << 23) | (rounded - 0x8000000u);
}
```

先拆出符号、阶码和尾数。阶码为 0xff 时原样返回；非规格化数直接用 fraction 计算 3/2，规格化数则补上隐含的最高位

1. 两种路径都按最近偶数舍入，规格化数还要处理尾数进位和阶码溢出。

P16 floatRoundEven

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exponent = (uf >> 23) & 0xffu;
    unsigned fraction = uf & 0x7fffffu;
    unsigned mantissa;
    unsigned shift;
    unsigned mask;
    unsigned remainder;
    unsigned half;
    unsigned integral;

    if (exponent == 0xffu)
        return uf;
    if (exponent < 126u)
        return sign;
    if (exponent == 126u) {
        if (fraction == 0)
            return sign;
        return sign | 0x3f800000u;
    }
    if (exponent >= 150u)
        return uf;

    shift = 150u - exponent;
    mantissa = 0x800000u | fraction;
    mask = (1u << shift) - 1u;
    integral = mantissa >> shift;
    remainder = mantissa & mask;
    half = 1u << (shift - 1u);
    uf = uf & ~mask;
    if (remainder > half || (remainder == half && (integral & 1u)))
        uf += 1u << shift;
    return uf;
}
```

NaN 和无穷原样返回；绝对值小于 0.5 时返回带符号的零，0.5 到 1 之间按中点规则处理，指数足够大时数值已经是整数。其余情况通过尾数低位计算余数，中点时查看整数部分奇偶性后决定是否进位。

P17 float_i2f

```
unsigned float_i2f(int x) {
    unsigned ux = x;
    unsigned sign = 0;
    unsigned bit = 0x80000000u;
    int exponent = 31;
    unsigned mantissa;
    unsigned shift;
    unsigned remainder;
    unsigned half;

    if (x == 0)
        return 0;
    if (x < 0) {
        sign = 0x80000000u;
        ux = ~ux + 1u;
    }
    while (!(ux & bit)) {
        bit >>= 1;
        exponent--;
    }
    if (exponent <= 23) {
        mantissa = (ux << (23 - exponent)) & 0x7fffffu;
    } else {
        shift = exponent - 23;
        mantissa = ux >> shift;
        remainder = ux & ((1u << shift) - 1u);
        half = 1u << (shift - 1u);
        if (remainder > half || (remainder == half && (mantissa & 1u)))
            mantissa++;
        if (mantissa == 0x1000000u) {
            mantissa >>= 1;
            exponent++;
        }
        mantissa &= 0x7fffffu;
    }
    return sign | ((exponent + 127) << 23) | mantissa;
}
```

负数先转成无符号绝对值，再查找最高有效位以确定指数。有效位不足 24 位时左移对齐，超过 24 位时截去低位并按最近偶数舍入；若舍入造成尾数进位，再同步调整指数。

P18 bitCount

```
int bitCount(int x) {
    int mask1 = 0x55 | (0x55 << 8);
    int mask2 = 0x33 | (0x33 << 8);
    int mask4 = 0x0f | (0x0f << 8);
    mask1 = mask1 | (mask1 << 16);
    mask2 = mask2 | (mask2 << 16);
    mask4 = mask4 | (mask4 << 16);
    x = (x & mask1) + ((x >> 1) & mask1);
    x = (x & mask2) + ((x >> 2) & mask2);
    x = (x & mask4) + ((x >> 4) & mask4);
    x = x + (x >> 8);
    x = x + (x >> 16);
    return x & 0x3f;
}
```

用 0x55、0x33、0x0f 掩码依次统计 2 位组、半字节和字节中的 1，再合并各字节计数。最后保留低 6 位，足以表示 0 到 32 的结果。

P19 bitReverse

```
int bitReverse(int x)
{
    int mask4 = 0x0f | (0x0f << 8);
    mask4 = mask4 | (mask4 << 16);
    int mask2 = mask4 ^ (mask4 << 2);
    int mask1 = mask2 ^ (mask2 << 1);
    int mask8 = 0xff | (0xff << 16);
    int mask16 = 0xff | (0xff << 8);
    x = ((x >> 1) & mask1) | ((x & mask1) << 1);
    x = ((x >> 2) & mask2) | ((x & mask2) << 2);
    x = ((x >> 4) & mask4) | ((x & mask4) << 4);
    x = ((x >> 8) & mask8) | ((x & mask8) << 8);
    x = ((x >> 16) & mask16) | (x << 16);
    return x;
}
```

依次交换 1 位、2 位、4 位、8 位和 16 位组，完成 32 位反转。每轮用对应掩码隔离移位结果；例如 0x80000004 最终变为 0x20000001。